The Astrophysical Journal Supplement Series, 277:63 (17pp), 2025 April https://doi.org/10.3847/1538-4365/adbdb6
© 2025. The Author(s). Published by the American Astronomical Society.
OPEN ACCESS

The Kratos Framework for Heterogeneous Astrophysical Simulations: Fundamental Infrastructures and Hydrodynamics

Lile Wang[1,2]
[1] The Kavli Institute for Astronomy and Astrophysics, Peking University, Beijing 100871, People's Republic of China; lilew@pku.edu.cn
[2] Department of Astronomy, School of Physics, Peking University, Beijing 100871, People's Republic of China
Received 2025 January 4; revised 2025 March 3; accepted 2025 March 4; published 2025 April 11

Abstract

The field of astrophysics has long sought computational tools capable of harnessing the power of modern GPUs to simulate the complex dynamics of astrophysical phenomena. The Kratos Framework, a novel GPU-based simulation system designed to leverage heterogeneous computing architectures, is introduced to address these challenges. Kratos offers a flexible and efficient platform for a wide range of astrophysical simulations, by including its device abstraction layer, multiprocessing communication model, and mesh-management system that serves as the foundation for the physical module container. Focusing on the hydrodynamics module as an example and foundation for more complex simulations, optimizations and adaptations have been implemented for heterogeneous devices that allow for accurate and fast computations, especially the mixed-precision method that maximizes its efficiency on consumer-level GPUs while holding the conservation laws to machine accuracy. The performance and accuracy of Kratos are verified through a series of standard hydrodynamic benchmarks, demonstrating its potential as a powerful tool for astrophysical research.

Unified Astronomy Thesaurus concepts: Astronomy software (1855); Computational methods (1965); GPU computing (1969); Hydrodynamical simulations (767); Hydrodynamics (1963)

1. Introduction

Astrophysical simulations are necessary to study a vast array of phenomena, from the intricate dynamics of planetary systems to the large-scale structure of the Universe itself. These simulations are inherently complex, requiring the resolution of multiple physical processes across an extraordinary range of spatial and temporal scales. To meet these challenges, computational astrophysics has long relied on advanced numerical methods and high-performance computing resources. The advent of GPUs has revolutionized the field of computational sciences, by offering the potential for substantial speedups in the computational performance. However, the actual design of GPUs, or heterogeneous computing devices as a kind, is *not* to simply speed up the computation task; instead, these devices actually speed up the overall computing throughput via adequately parallelizing the computing tasks.

In this context, Kratos is introduced as a novel GPU-optimized simulation system designed to address the challenges of heterogeneous simulations in astrophysics and beyond. Kratos should represent the significant advancement in the field that has been emerging in recent years until the near future, offering a unified framework that supports a wide range of physical modules, including hydrodynamics, consistent thermochemistry, magnetohydrodynamics, self-gravity, radiative processes, and more. A critical aspect in the development of Kratos is its performance optimization for heterogeneous devices while maximizing the flexibility. Because the architectures of GPUs are special from the hardware to the programming model, detailed designs are necessary for the infrastructures on which those modules are implemented. Their design philosophy centers on achieving high performance without compromising accuracy, ensuring that they remains a versatile tool for a diverse array of astrophysical applications.

Kratos is among the handful of astrophysical fluid dynamics codes that attempt to leverage the power of GPUs, but it is neither the first nor the sole one. Prior works, such as P. Grete et al. (2021) for K-Athena and J. M. Stone et al. (2024) for AthenaK, have detailed the integration of MHD solvers within Athena++ using Kokkos (C. R. Trott et al. 2022). G. R. J. Lesur et al. (2023) have introduced IDEFIX as a Kokkos-based adaptation of the PLUTO code. Several other astrophysical fluid dynamics codes also utilize GPU-accelerated computing through platform-specific tools like CUDA, including Ramses-GPU (P. Kestener 2017), Cholla (E. E. Schneider & B. E. Robertson 2015), and GAMER-2 (H.-Y. Schive et al. 2018). Nevertheless, Kratos continues to pursue outstanding performance by directly applying optimizations to low-level features that are closer to heterogeneous hardware, while maintaining flexibility through the use of modern computational programming practices. It is designed as a general-purpose framework for multiple physical modules, with the hydrodynamic modules detailed in this paper as the starting point.

Kratos is composed in modern C++. Instead of building on existing high-performance heterogeneous computing libraries that operate on higher levels (such as Kokkos), Kratos complies with the C++-17 standard and the corresponding standard library, and constructs the framework directly on the GPU programming model, e.g., CUDA for NVIDIA GPUs (NVIDIA 2024) and Heterogeneous-Compute Interface for Portability (HIP) for AMD GPUs and the derivatives,[3] by composing an abstraction layer with zero runtime overhead to insulate the actual implementation of algorithms from the subtle details and nuances existing in different programming

Original content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence. Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.

[3] https://rocm.docs.AMD.com/projects/HIP/en/latest

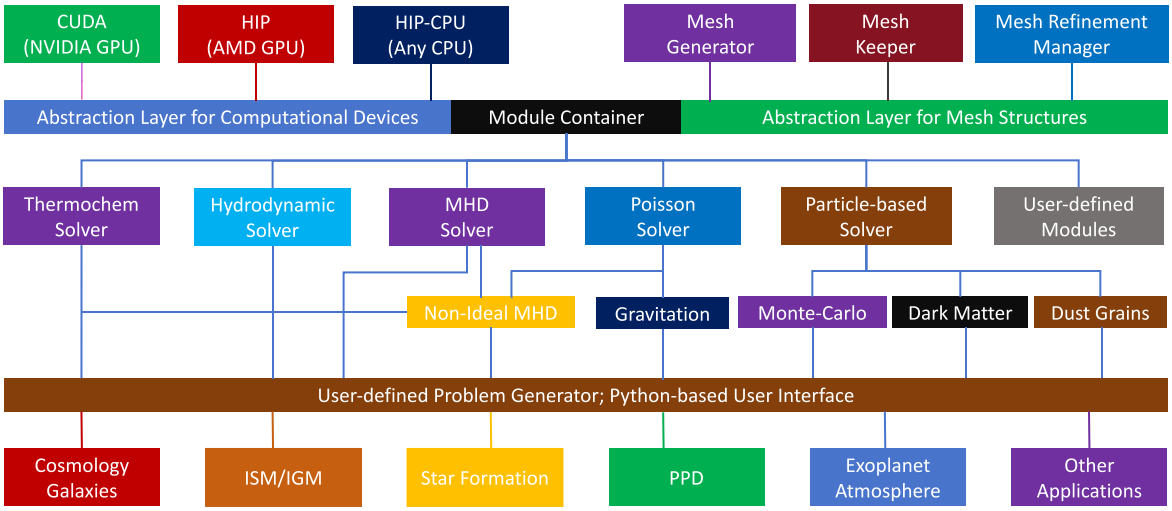


Figure 1. The overall structure of the Kratos code. Based on the abstraction layer for computing devices and the abstraction layer for the mesh structures, the module container holds all capable modules for physical processes taking place in astrophysical simulations (hydrodynamic module elaborated in this paper, other modules will be described in forthcoming papers; L. Wang 2025, in preparation). While complicated physical calculations can be constructed through the interactions of modules, a user interface (problem generator) handles the definition of initial, boundary, and in-simulation conditions. Such interface can also handle the initial conditions defined by a Python-based interface. Actual astrophysical objects and processes are simulated based on all fundamental systems described above.

models. This approach guarantees the compatibility with different kinds of heterogeneous and conventional devices made by most mainstream manufacturers. With the help of proper utilities (e.g., HIP-CPU[4]), Kratos also maintains the compatibility with the "conventional" CPU architectures for developing purposes, and can be extended to actually operate on these architectures when necessary. Meanwhile, the higher-level encapsulations (including Kokkos) typically focus on the most generic compatibility and portability, which could cause significant performance impact due to the inability to fully include the low-level features that are not available on all supported platforms. In contrast, the algorithms directly implemented on low-level features in Kratos are necessary to optimize the code performance by maximizing the utilization of various heterogeneous features, especially detailed dispatching of computing streams and threads, as well as the shared memory allocated on the caches of multiprocessors (see, e.g., Section 3). This approach also enables deep utilization of template metaprogramming to maintain a high-level modularity and maximize the flexibility and capability of Kratos as a platform, on which more physical modules are to be included (detailed in incoming papers). Kratos itself is based on the basic programming models (e.g., CUDA or HIP) and the standard Message Passing Interface (MPI) implementations. Otherwise, it is fully self-contained, rather than depending on third-party libraries (including the input and output modules, and the subsequent data reading and processing programs in Python), in order to maximize the compatibility and ease the deploying procedures on different platforms.

Before this paper describing the methods, Kratos has already been used (after thorough tests) in several astrophysical studies (e.g., L. Wang & X. Li 2022; A. Lv et al. 2024), and multiple further applications have also been carried out in various astrophysical scenarios. Multiple fundamental modules, including the input parameter parser, binary inputs and outputs module, and the device abstraction layer, have also been used and made publicly available in other codes composed or contributed by the author (e.g., S. Wang et al 2025). This paper, describing the fundamental infrastructures for all modules implemented on Kratos for various astrophysical systems, also serves as the foundation of several forthcoming papers on the method and modules, including MHD, thermochemistry, and particle-based simulations (L. Wang 2025, in preparation).

The structure of this paper is as follows. A detailed description of the basic infrastructure in Section 2 is followed by the elaboration on the implementation of the hydrodynamics module in Section 3 as an example of general modules. A comprehensive set of code tests, including accuracy, symmetry, and performance optimizations, are included in Section 4. Finally, Section 5 summarizes the paper, and outlines the incoming works for multiple physical mechanisms based on the Kratos framework.

2. Basic Infrastructures

The fundamental infrastructures of the Kratos code are briefed in Figure 1. One of the most apparent features it holds is the separation of the mesh structures from the physical modules. In general, the mesh structure subsystem only contains the geometry of the mesh, including the overall mesh tree on which the subgrids ("blocks" hereafter) are allocated. The physical modules are included in a separate module container, which has similar functionality as the "task lists" in many other numerical simulation systems (e.g., J. M. Stone et al. 2020), and each module has an internal "task list" of its own. The execution sequence can be designed by users in each cycle of Kratos runs for better flexibility and compatibility.

2.1. Abstraction Layer for Devices

In what follows, the paper uses the term "host" to denote the hardware and operations on the CPU (including the RAM on the CPU side), and "device" for the hardware, software interfaces, and instructions that are heterogeneous to the CPUs (e.g., GPUs; the term "GPU" will be used to denote all heterogeneous devices with similar architectures in what follows). On the contemporary high-performance and consumer-level computing markets, there

[4] https://github.com/ROCm/HIP-CPU

have already been multiple programming models available, which are usually specific to the manufacturers of devices. To extend the portability on various hardware and software platforms while maximizing the performance by keeping the code close to the hardware, Kratos decides to implement the computations and algorithms via a new pathway by formulating *all* device-side codes in the subset that is the intersection between HIP and CUDA syntaxes, avoiding the usage of any manufacturer-specific device calls.

On the host side that dispatches data transfer and device function (namely, "kernels") launching, the interfaces could have subtle differences (e.g., cudaMallocHost versus hipHostMalloc for allocating nonpageable host memory spaces). To resolve similar issues, Kratos also holds an abstraction layer on both host and device sides. This abstraction layer is implemented as a C++ class, containing the following ingredients that encapsulate the differences in syntax across various programming models:

1. initialization and finalization (destruction and resource release) for device environments;
2. device dispatching that assigns a specific device to a CPU process;
3. fundamental attributes, including __global__, __host__, __device__, etc., which are commonly used in almost all relevant programming models, including CUDA and HIP;
4. directives related to shared memory on the device side;
5. application interfaces, including memory space allocations and deallocations, host-device data transfer and copy procedures, calls of device calls (namely, "device kenel launching"), and GPU stream and event operations.

All nuances comparing different programming models are absorbed by the abstraction layer, which assures Kratos be compiled and run on different platforms without any modifications to the code. Meanwhile, features that are particularly crucial for GPUs, such as shared memory allocation and access, constant memory access, as well as stream and event operations, remain readily accessible. It is noticed that the intermediate layers have been introduced to models other than HIP and CUDA, including oneAPI[5] and OpenCL 3.0.[6] Equipped with encapsulation models such as chipStar,[7] codes based on HIP can also be compiled and run on systems that are compatible with oneAPI and OpenCL 3.0. What is more, using the open-source library HIP-CPU that emulates GPUs via the thread building block (TBB) library, Kratos simulations can also be conducted using CPU with various architectures that are compatible with TBB, including x86, ARM, and RISC-V.

2.2. Multiprocessing Communication Model

Kratos can be operated on multiple devices in parallel, and the code design requires that each device is controlled and managed by one host-side process. Data transfer among devices and processes is handled by the communication module, which is established on the MPI by default, while other implementations are also possible. It is worth noting that several manufacturers have introduced "device-aware MPI," which allows direct transfers of device-side data without passing through the host-side memory. In general, however, Kratos does *not* always utilize this "device-awareness" MPI feature, because of the absence of "streams" in the MPI interfaces, which disables asynchronous communication.[8] In typical simulations involving multiple blocks (or submeshes; see Section 2.3.1), communication operations can be launched immediately once each block is ready, without the need to wait for the completion of processing all blocks on a single GPU. The asynchronous pattern is thus essential for all possible efforts to effectively "hide" communication operations behind the computations of other blocks. Consequently, Kratos has chosen to develop its own independent communication module on which the mesh-management system is constructed, instead of relying on existing adaptive mesh-refinement frameworks such as AMReX (W. Zhang et al. 2019).

Kratos employs a unified and encapsulated multiprocessing communication module, which adopts the standard MPI implementation as its default "backend," adhering to version 3 of the standard. The module interfaces are largely similar to the MPI application programming interface (API) in the C programming language, with an extra feature to optimize device-to-device communications: interfaces such as nonblocking send and receive operations are designed to accept device streams as arguments. This allows a communication task to be queued on a specific device stream, ensuring the correct handling of data dependencies and execution sequence. These operations inevitably involve data transfer between host and device memories, which causes extra time consumption of data transfer. The introduction of device stream, however, enables the procedure pattern that allows the data transfer to take place following the correct sequence after all prerequisite computations are accomplished. By properly designing the pattern of communication procedures, Kratos generally attempts to maximize the efforts of "hiding" the communication operations behind the computations, and thus improving the overall performance of multidevice computation. Note that such a communication feature is unlikely to be implemented even with the device-aware MPI (such as the "CUDA-aware" or "HIP-aware" MPI implementations that allow direct communications between device-side memory spaces), due to the absence of device streams in device-aware MPI API parameters. Details of the design for asynchronous communications utilizing streams are described in the explanation of each model, including the standard hydrodynamic module elaborated in Section 3.

The special communication module requires host-side buffers for proper data transfer, which are designed to be allocated on the first invocation. Capacities of these buffers are given by multiplying the requested size by a safety factor, which is typically around $\sim$1.2. This factor has been found to be effective in the majority of multiple scenarios, although it should be noted that the optimal value may differ depending on the specific application requirements. These buffers are intentionally retained rather than being released back to the system, unless the subsequent requests for buffer sizes exceed their current capacity (when the original buffer is released and replaced with a newly allocated one), or until intentional deallocation (e.g., one simulation as a whole comes to an end). Note that, although the hydrodynamic solver described in this paper does not involve variable buffer sizes, this feature is still

[5] https://www.intel.com/content/www/us/en/docs/oneapi/programming-guide/2025-0/overview.html
[6] https://github.com/KhronosGroup/OpenCL-Guide
[7] https://github.com/CHIP-SPV/chipStar
[8] https://developer.nvidia.com/mpi-solutions-gpus

required by other modules (especially particle modules) elaborated in following papers. This design choice minimizes the time-consuming overhead of buffer allocation and deallocation, thereby maintaining a clean and efficient runtime environment.

2.3. Mesh Manager

2.3.1. Mesh Tree and Refinement

Kratos utilizes 2^d-trees to manage its structured mesh, where $d \in \{1, 2, 3\}$ (thus octree in 3D, quadtree in 2D, and binary tree in 1D). The trees fulfill dual roles: (1) to identify the spatial configuration of the subgrids, and (2) to facilitate the allocation of geometry necessary for mesh refinement. Each node within this tree (not to be confused with the server node that denotes a single set of computers) is referred to as a "block." These blocks are the foundational elements that collectively compose the entire mesh, covering the entire domain of the simulation.

Based on the tree structure, mesh refinement is implemented for Kratos. Each block is represented by a node in the tree. Once a block is to be refined, it is replaced by 2^d blocks that have a doubled resolution and halved size. When the number of refinement level L is greater than 1, the 2^d-tree refinement scheme is carried out recursively, with an extra procedure adding prospectively extra refinement regions to guarantee that the absolute value of the difference in L is no more than 1 for neighboring blocks.

The tree structure is organized by the C++ standard template library member std::map with the four-integer key values allocated in lexicographical order. These four-integers, indicated as $\{I, J, K, L\}$, define the "block indices": the first three (I, J, K) stand for the logical indices of the block, and the L stands for the refinement level (the coarsest level has $L = 0$). When launched in parallel, each process holds a copy of the basic tree that holds the key values and communication information (including the identity number and the process rank number) of all blocks, while the detailed geometry data are only filled into the nodes that are held by the current process. This approach guarantees that all neighbors of every block can be easily identified for communication purposes, while the tree size and communication costs are minimized when the tree is updated.

2.3.2. Job Dispatching and Load Balancing

When Kratos simulations are carried out on multiple devices, the computation load is dispatched to every device involved based on the mesh tree structure. The job dispatching is managed by the load balancing module, which requires a scheme that maps the blocks distributed in the d-dimensional space onto a 1D line segment. Using this mapping scheme, blocks are eventually distributed on the computing devices using the device number, either evenly or by a user-defined array of weights that takes other factors into account (e.g., the actual computing capability of each device). Although Kratos allows users to enroll their own implementation, by default, there are two different load balancing modules that can be selected from.

The first is the row-filling scheme, which maps the (I, J, K) block indices onto 1D lexicographically. When the 2^d-tree mesh refinement is applied, the simple row-filling scheme is applied to the blocks in the refined region, and replaces the "original" coarse block in place. The row-filling scheme addresses simulations in simple geometric layout with efficiency very close to optimal schemes, and suits several situations even better than other schemes when the row-filling scheme minimizes cross-process data dependencies (e.g., in simulations with plane-parallel ray tracing).

The second method is based on fractal space-filling curves, which naturally resolves the ordering of the refined region, since these curves can be generated recursively to a finer level once refinement is applied. Among the possible selections, the Hilbert curves usually perform better for several reasons. The Hilbert curves are 2-based, in contrast to some other famous choices (e.g., the 3-based Peano curves), which maximizes the flexibility of the system. In addition, the Hilbert curve guarantees that the neighbors on the mapped 1D space are also neighbors in the d-dimension space, which excels over other possibilities, e.g., Z-ordering curves. To maximize the flexibility, Kratos also allows users to adopt their own rules for generating space-filling curves under the Lindenmayer system (L-system for short), and the default rules for Hilbert curves are also implemented under the L-system for 2D and 3D, respectively. In 3D, the L-system for Hilbert curves reads

$$\begin{aligned} X \rightarrow & \wedge < XF \wedge < XFX - F \wedge > > \\ & \times XFX \,\&\, F + >> XFX - F > X - >; \\ & X \Rightarrow \text{Axiom (recursion interface)}, \\ & F \Rightarrow \text{Move forward}, \\ & + \Rightarrow \text{Yaw angle } + \pi/2, \\ & - \Rightarrow \text{Yaw angle } - \pi/2, \\ & \wedge \Rightarrow \text{Pitch angle } + \pi/2, \\ & \& \Rightarrow \text{Pitch angle } - \pi/2, \\ & > \Rightarrow \text{Roll angle } + \pi/2, \\ & < \Rightarrow \text{Roll angle } - \pi/2 \quad . \end{aligned} \tag{1}$$

The application of space-filling curve load balancing will be presented when illustrating the examples of hydrodynamics (Section 4.7).

2.4. Modules and Module Manager

Physical processes within a simulation are computed by specialized physical modules managed by the module container, which in turn relies on the data provided by the mesh manager. Unlike most other simulation systems, the relationship between the module container and the mesh manager is unidirectional in Kratos: the mesh does not hold any data or information about the modules, while the physical modules extract the necessary geometric data from the mesh to perform their calculations. When simulations are distributed across multiple devices, interblock communication often becomes necessary for different physical modules to function effectively. Since each module may have unique communication requirements and patterns, the modules handle all related data and communication procedures independently. The data needed for these computations are stored within the modules themselves. This approach maximizes the flexibility of Kratos, which allows it to work as a code defined *fully* by the modules involved—for example, although hydrodynamics is the foundation of many modules, Kratos can still be a radiative transfer code that is totally irrelevant to the dynamics of fluids (H. Yang & L. Wang 2025, in preparation).

In scenarios where data must be shared among multiple modules, explicit specification of data coupling is employed, facilitated through a data proxy scheme. This scheme involves

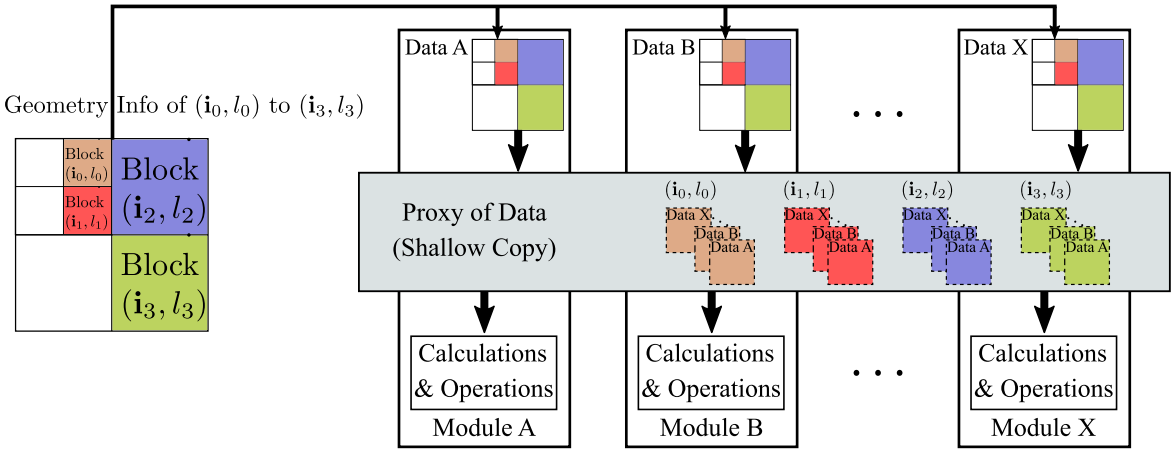


Figure 2. The method that Kratos handles the interaction of different physical processes via sharing the data held by modules. An instant data structure, encapsulating the shallow copy of all data needed held by all relevant modules, is created as a data proxy right before the launching of each GPU job conducting the calculations.

creating a "shallow copy" of the data, which includes only the essential geometries and pointers to the GPU memory, on the CPU side before the module is executed on the GPU side. This method is depicted in Figure 2, and the implementation details of the modules will be discussed within the context of the modules themselves, providing a clearer understanding of how they operate and interact within the system.

The modules included with Kratos are intentionally crafted to be user accessible for modification and expansion. The typical approach to extending their functionality is through inheritance, where users create custom classes by inheriting from the base default C++ class and overriding the relevant member functions. However, it is important not to implement this overriding using runtime polymorphism via virtual functions in C++. According to the tests conducted by the author in parallel to the developing procedures, the usage of virtual tables can often lead to significant performance degradation (by $\sim 10^2$ times) on most heterogeneous devices,[9] with various tests indicating a decrease in performance by more than 1 order of magnitude (not detailed in the current paper). Moreover, there is generally no practical need for runtime polymorphism in simulations, as the methods and algorithms used are typically predetermined. Consequently, Kratos opts for a different strategy, the compilation-time polymorphism through template metaprogramming. This approach utilizes the curious recursive template pattern (see also J. O. Coplien 1995), which is based on F-bounded polymorphism (see P. Canning et al. 1989). By employing this pattern, Kratos can achieve the desired flexibility and customization without incurring the performance penalties associated with runtime polymorphism, thus ensuring that the modules remain efficient and effective for simulation tasks.

3. Hydrodynamics: The Example and Foundation of Many Modules

The default hydrodynamics module in Kratos is not only the first implemented example module but also the most optimized one. It plays a pivotal role in Kratos, as hydrodynamics is a fundamental aspect of various astrophysical research disciplines. Moreover, it serves as the cornerstone for numerous other modules and applications, such as magnetohydrodynamics, reacting flows, radiative hydrodynamics, and cosmology, among others. While detailed discussions of these modules and applications will be presented in subsequent papers, the focus of this work is to delineate and clarify the adaptations and optimizations made for heterogeneous devices. These optimizations are also the methodological foundation for the forthcoming studies.

The default hydrodynamic module is constructed within the Godunov framework, solving the Euler equation on discrete mesh,

$$\begin{aligned} &\partial_t \rho + \nabla \cdot (\rho \boldsymbol{v}) = 0; \\ &\partial_t (\rho \boldsymbol{v}) + \nabla \cdot (\rho \boldsymbol{v}\boldsymbol{v} + p\boldsymbol{I}) = 0; \\ &\partial_t \epsilon + \nabla \cdot [(\epsilon + p)\boldsymbol{v}] = 0, \end{aligned} \tag{2}$$

where ρ, p, and $\boldsymbol{v}$ are the fluid mass density, pressure, and velocity, $\epsilon = \epsilon_g + \rho v^2/2$ is the total energy density (ϵ_g is the gas internal energy density), and ∂_t denotes time derivatives. The solution procedures can be broken down into three key "submodules": (1) the slope-limited piecewise linear method (PLM) reconstruction, (2) the Harten–Lax–van Leer with contact surface (HLLC) Riemann solver, and (3) the Heun time integrator, which is one of the second-order Runge–Kutta methods. When mesh refinement is employed, it is crucial to implement specialized schemes at the boundaries between finer and coarser mesh levels. The algorithms that govern the fluxes across cell surfaces are mathematically equivalent to those found in most existing computational fluid dynamics codes, as referenced in E. F. Toro (2009). However, to achieve optimized performance, it is essential to tailor these algorithms specifically for heterogeneous devices via rearranging the submodules and schemes summarized in Figure 3. The synchronization calls of streams, which mark and guarantee that all operations are finished in sequence prior to the call on the same stream, are "postponed" to the communication. Because there is *no* dependency among different streams until synchronizations, such a pattern allows the actual data transfer (from the GPU to the CPU side, and from one process to another) to take place in the background while the GPU is handling the prerequisite calculations for other blocks. In this way, a relatively large fraction of the communication costs are hidden behind the computation, so that the communication time is minimized. Detailed elaborations of more optimizations will follow in the subsequent discussion.

[9] Typically, this performance impact is attributed to the virtual tables allocated on the global graphics memory on GPUs, which could suffer from severe latency when applied.

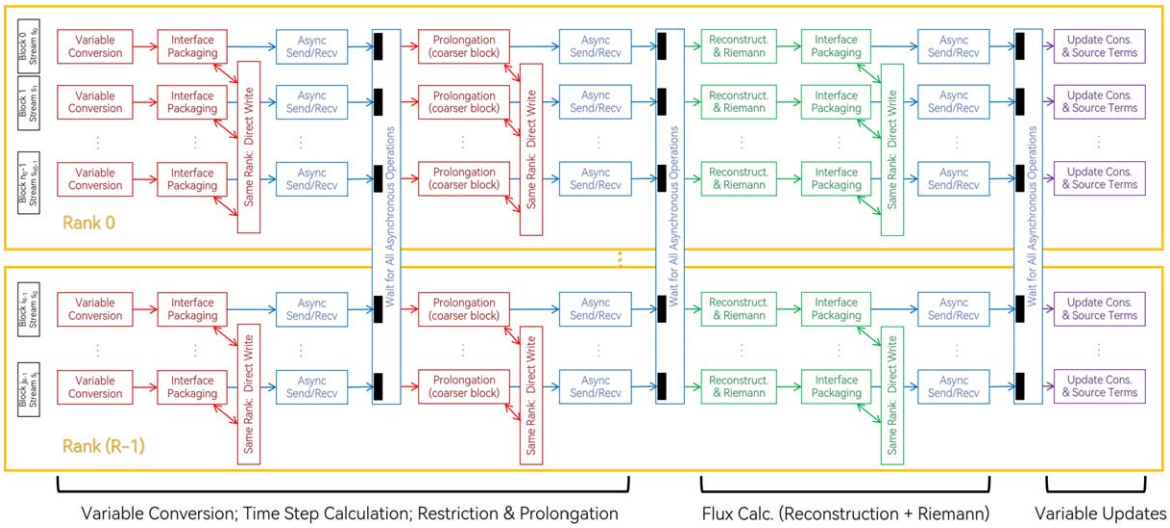


Figure 3. Scheme for one hydrodynamic cycle in Kratos, showing the parallelization pattern and the interface calls for data exchange. Each "rank" denotes a process, typically controlling one GPU, on which the calculations for multiple blocks are conducted. Each row (labeled by the "Block X, Stream s_X" at the beginning of each row) denote the numerical processes of one block on the 2^d-tree mesh, which are located on one GPU stream for proper data synchronizations. Different steps of the Godunov solver are marked by colors (red for the preparation of primitive variables, green for calculation of the Riemann fluxes, and magenta for the updates of conservative variables), while the communication operations are marked with blue. Black solid blocks stand for the synchronizations of each stream, taking place within the calls of communication operations to minimize waiting and thus maximize the efficiency.

3.1. Variable Conversion and Boundary Conditions

The default Riemann solvers in Kratos hydrodynamics work with primitive variables—for hydrodynamics, the variable set contains (ρ, p, $\boldsymbol{v}$). As the conservative variables are the basic ones whose evolution is calculated on each step, conversion from conservative to primitive variables needs to be calculated first. The length of time step Δt is also calculated in the meantime of such a variable conversion based on the Courant–Friedrichs–Lewy conditions, which is conducted only on the first substep when the time integrator adopts a multistep method.

If a Godunov algorithm has nth-order spatial accuracy, the reconstruction step will require n cells on both sides for each cell interface. The consistency of calculations requires primitive variables to be available in the ghost zones, which form "buffer layers" surrounding blocks. This paper refers to the schemes of setting physical variables in the ghost zones as "boundary conditions," which include both "physical boundaries" (p-boundaries) for the actual boundary of the whole simulation domain, and "communication boundaries" (c-boundaries) for the interfaces between neighboring blocks.

3.1.1. Physical Boundary Conditions

Kratos allows users to enroll their own p-boundary conditions to accommodate for the actual situations of astrophysical simulations. Note that, when periodic boundary conditions are applied to the domain, the domain boundaries become c-boundaries instead. On the programming side, the p-boundary module in Kratos conducts proper manipulations including the reflection and alternation of coordinate axes during the loading and saving stages of the variables inside and near the p-boundaries, so that all actual p-boundary operations are mathematically equivalent to dealing with the boundary condition in the x-direction on the left, as is shown in Figure 4.

Users can set different p-boundaries on different sides. There are three default options, which are expressed with the indexing

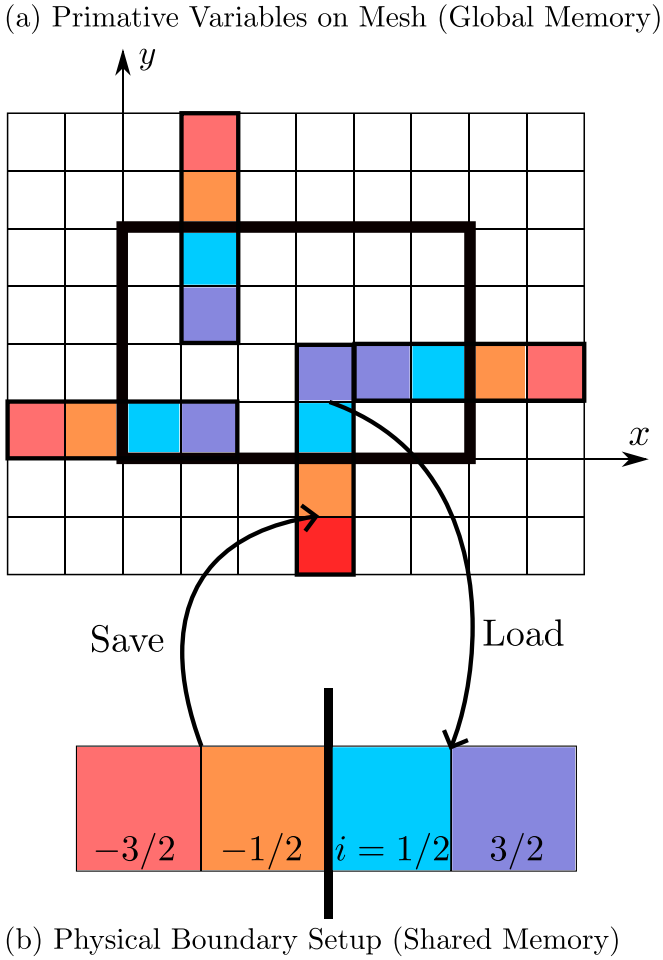


Figure 4. Boundary condition methods for hydrodynamics, showing four normal columns as examples to illustrate how Kratos rotate the normal columns to the x-direction to unify the methods on the interfaces of block boundaries. Colors indicate the corresponding cells before and after the rotations.

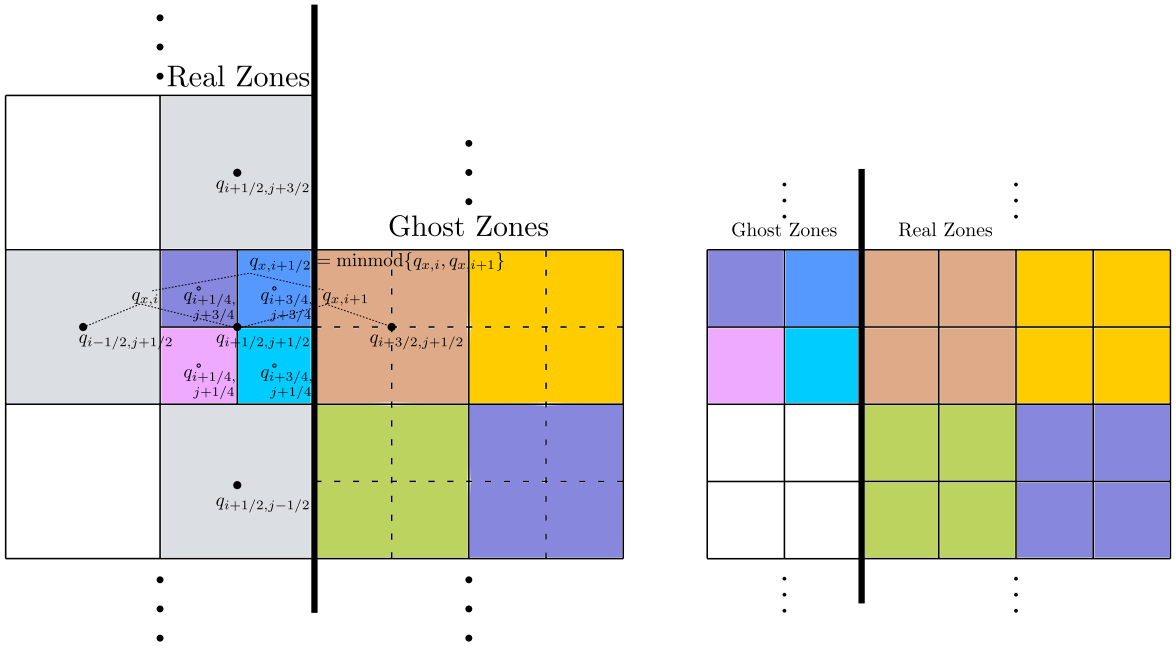


Figure 5. Restriction and prolongation operations near the block refinement interface. Colors are used to mark the corresponding relations across the interface separating coarser (left) and finer (right) blocks. The prolongations are computed on the coarser side, and the x-derivatives $q_{x,I,J}$ are calculated using minmod slope limiter in a pattern indicated on the coarser mesh (derivatives along other directions can be obtained similarly).

for the left x-boundary (see also Figure 4; integers are used for cell faces, and half integers for cell centers):

1. free boundary, $q_{-i-1/2} = q_{1/2}$;
2. outflow boundary, $q_{-i-1/2} = q_{1/2}$, but $v_{x,-i-1/2} = \min\{v_{x,1/2}, 0\}$;
3. reflecting boundary, $q_{-i-1/2} = q_{i+1/2}$, but $v_{x,-i-1/2} = -v_{x,i+1/2}$.

Here, $0 \leqslant i \leqslant n_{\rm gh} - 1$, and $n_{\rm gh}$ correspond to the spatial order of the reconstruction. Note that the transform automatically rotates the normal component of velocity (pointing inwards) into v_x, and rotates v_x back to the desired component and direction after the boundary condition algorithms are applied.

3.1.2. Communication Boundaries

When a simulation domain is partitioned into multiple blocks, it is crucial to establish consistent c-boundaries for the physical variables across these blocks. In the context of hydrodynamics, c-boundaries pertain only to the interfaces between blocks, or block faces. The communication patterns between these blocks are relatively simple and direct, especially when there is no mesh refinement involved. To further reduce the time spent on communication, Kratos incorporates an additional optimization technique, handling data transfers within the same device by directly writing the ghost zone data to the destination, bypassing the usual communication interfaces as described in Section 2.2.

Kratos hydrodynamics relies on the 2^d-tree method also for its mesh refinement, which plays a pivotal role in managing the c-boundaries at the interfaces between refined and coarser meshes. The general mechanism of c-boundary mesh refinement is presented in Figure 5, which illustrates the prolongation and restriction schemes for primitive variables near the block boundaries that separate blocks of different levels. The restriction operations, which involve transferring data from finer to coarser grids, are simply executed by performing volume-weighted averages. Conversely, the prolongation step, which interpolates data from coarser to finer grids, requires a more nuanced approach. This step must comply with the total variance diminishing condition to prevent numerical instabilities that can arise from the interpolation process. The minmod function,

$$\text{minmod}(a, b) = \begin{cases} a, & |a| < |b| \quad \text{and} \quad ab > 0; \\ b, & |a| \geqslant |b| \quad \text{and} \quad ab > 0; \\ 0, & ab < 0, \end{cases} \tag{3}$$

is used to evaluate the derivatives in the slope-limited PLM. The minmod function is crucial for maintaining the stability and accuracy of the simulation by limiting the slope of the reconstructed variables, thus preventing oscillations that could lead to unphysical results.

The communication pattern in Kratos hydrodynamics is designed in a specific sequence to ensure the consistency and efficiency of computations:

1. finishing all communication operations that are not directly related to cross-level block boundaries, to prepare for necessary data for prolongation;
2. conducting prolongation calculations on the coarser blocks over all cross-level boundaries before sending the prolonged data to finer blocks, as the coarser blocks have complete information required;
3. calculating the fluxes on all blocks (see Section 3.2);
4. sending the flux information at the cross-level block boundaries from the finer side to the coarser side, and summing up the fluxes accordingly to obtain the fluxes that are actually adopted on the coarser side at the cross-level interface, so that the conservation laws are maintained at the c-boundaries.

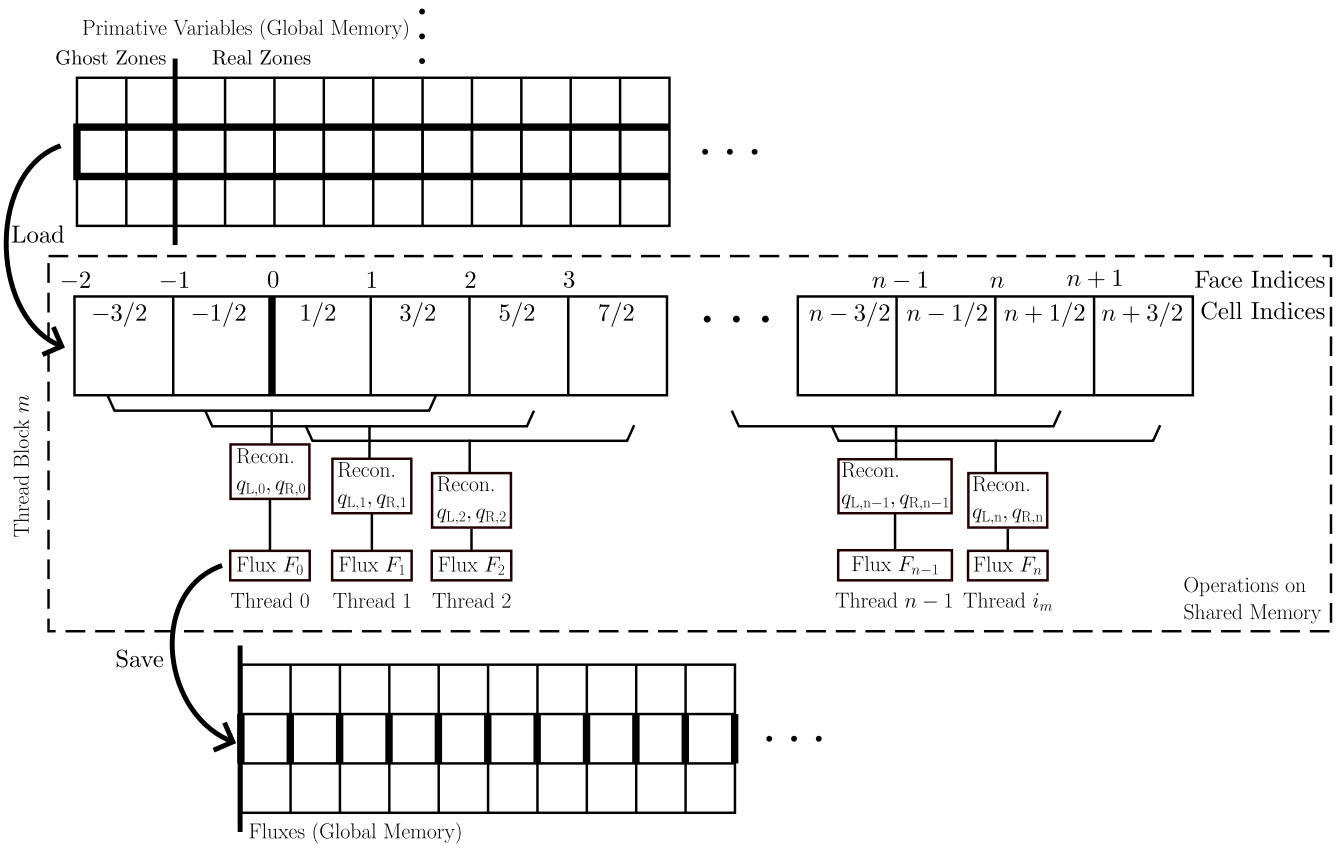


Figure 6. Patterns of calculating the fluxes for hydrodynamics, illustrating the utilization of shared memory. All processes within the middle dashed box use the shared memory instead of the global memory, as the memory-related operations are frequent, and the data in each cell is typically used many times by multiple threads (indicated by the braces and blocks in the middle box).

3.2. Flux Calculation

Almost all hydrodynamic methods in the finite volume method category have the most time-consuming part on the computations of fluxes for hydrodynamic variables. The efforts of improving the performance of hydrodynamic simulations should focus on the optimization of flux calculations.

3.2.1. Shared Memory Utilization

The access of data in the global device memory[10] ("global memory" hereafter; not to be confused with the system memory on the CPU side) suffers from excessive latency (typically equivalent to $\sim 10^2$ or even more clock cycles for the awaiting computing units) on heterogeneous devices. This issue becomes even worse for transposed data access when calculating the fluxes along the directions perpendicular to the x-axis. To reduce the cost of memory access, almost all contemporary heterogeneous computing devices and their corresponding model have an important feature, named "shared memory," that can significantly improve the performance. Shared memory indicates memory spaces that are typically allocated on the L1 cache of stream multiprocessors. Such memory spaces are equivalently fast as L1 when accessed by computing units, while the data flow can be controlled by the programmer, in contrast to the ordinary L1 cache spaces that are controlled by the default dispatcher.

Proper utilization of the shared memory is a necessity for maximizing the performance of flux calculations, minimizing the frequency of accessing the data staying in the global memory by adequately coordinating the data flow. The reconstruction step, regardless of the actual algorithm, obtains the physical variables on both sides of an interface separating neighboring cells. The lth order reconstruction method will require data from $2(l+1)$ cells for one interface, and in turn, data in one cell (not in ghost zones) will be read by the reconstruction of $2(l+1)$ interfaces. These data dependencies, illustrated in Figure 6, clearly indicate the dispatching procedures for reconstruction, to load the row of related data into the shared memory first, and then conduct subsequent calculations using the data and space in the shared memory.

3.2.2. Mixed-precision Methods

In the Kratos framework, the hydrodynamics module is implemented via a mixed-precision approach, a feature that ensures both accuracy and computational efficiency. This methodology defines a `float2_t` data type (double precision by default, typically represented by 64-bit floating-point numbers), to maintain the conservation laws of hydrodynamics with machine-level precision for conservative variables, encompassing mass density, energy density, and momentum density. Meanwhile, the hydrodynamics module approximates the fluxes of these conservative variables using approximate Riemann solvers, a common practice in most of the finite volume hydrodynamics simulation codes. Therefore, employing full-double-precision methods for these calculations is not

[10] Typically, the memory spaces whose sizes are marked explicitly by manufacturers as "graphics memory."

mandatory, provided that numerical stability is guaranteed. To this end, the hydrodynamics module opts for the `float_t` data type, which defaults to single precision (typically represents 32-bit floating-point numbers), for the reconstruction schemes and the approximate Riemann solvers. This approach ensures that the conservation laws are upheld to the precision inherent in `float2_t`, while also maintaining physical reliability by holding the conservation laws.

It is worth noting that, through the appropriate selection of compilation options, the data types `float_t` and `float2_t` can be mapped to either single or double precision, offering flexibility in the precision requirements of the simulation. This feature is useful in scenarios where double precision is mandated, especially when using devices that have comparable computational capabilities for both single- and double-precision operations (e.g., the Tesla series of computing GPUs).

The process of resolving fluxes is generally broken down into two main steps. The initial step involves the slope-limited PLM reconstruction, which is easily executed in single precision. However, it is crucial to use the cell center distance, Δx, directly in the denominator for numerical derivatives to avoid significant errors that can arise from large number subtractions, a phenomenon known as "catastrophic cancellation." The subsequent step involves approximately solving the Riemann problem, a process similar to the methods detailed in E. F. Toro (2009). The HLLC solver is used as a default example, where the contact surface speed s_* is the crucial value,

$$s_* = \frac{[p_R - \rho_R u_R(s_R - u_R)] - [p_L - \rho_L u_L(s_L - u_L)]}{\rho_L(s_L - u_L) - \rho_R(s_R - u_R)}. \quad (4)$$

It is noted that, when $|s_i - u_i| \ll |u_i|$ ($i \in \{L, R\}$) for either left (L) or right (R) states, the reliability of the equation for s_* is compromised. Numerical tests have shown that single-precision calculations can lead to more instances of catastrophic cancellations and unreliable results, or even vanishing denominators in the equation for s_*. To mitigate this, subtraction-safe modifications are implemented,

$$\begin{aligned} \Delta s_L &\equiv s_L - u_L = -c_{s,L} q_i, \\ \Delta s_R &\equiv s_R - u_R = c_{s,R} q_i; \\ q_i &= \left[1 + \frac{\gamma+1}{2\gamma}\left(\max\left\{\frac{p_*}{p_i}, 1\right\} - 1\right)\right]^{1/2}, \end{aligned} \quad (5)$$

leveraging the fact that Δs_L is always negative, and Δs_R is always positive, which prevents catastrophic cancellations or vanishing denominators in Equation (4).

By applying a mixed-precision algorithm to the HLLC Riemann solver within the Kratos module, extensive testing has confirmed that the relative error remains below 10^{-3} when compared to the double-precision version across the parameter space relevant to typical astrophysical hydrodynamic simulations (not shown in this paper). For direct tests against analytic solutions in shock tube scenarios as part of the Godunov solver pipeline, readers are directed to Section 4.2 for further details.

3.3. Time Integrator

The time integrator is an independent submodule within the hydrodynamic solver. By default, the time integration is conducted by the Heun method, one of the widely adopted second-order Runge–Kutta integrators, in which the "one-step integration" block is equivalent to the whole cycle described in Figure 3. One feature of the Heun method is that the integrated variables of the predictor substep (also known as the intermediate substep) directly enter the results of the whole step with the weight $1/2$, which requires that the predictor substep results should be recorded with the same precision as the whole step.

When the block refinement conditions are complicated, the Kratos system does not prescribe a universal time stepping scheme. This hydrodynamic module adopts global time stepping by default, while users are still allowed to include localized, block-specific time stepping schemes. In a wide variety of astrophysical scenarios, no more than ∼six levels of mesh refinement are typically adopted, including young stellar accretion disks, turbulent interstellar media, circumgalactic and intergalactic media, etc., where the global time stepping scheme is already sufficient to support most scientific exploration goals. Kratos has also planned block-level adaptive time stepping that is emerged into the mesh-refinement framework. Because the current framework already involves asynchronous communication (Section 2.2) and load balancing based on space-filling curves (Section 2.4), proper weighting (which could be as simple as weight each block by the number of equivalent time steps on each global cycle) can resolve the issues associated with block-level adaptive time stepping, and will be implemented when necessary.

When the memory occupation is a concern, one can also select the memory-optimized second-order van Leer integrator. This integrator can store the prediction substep results in the space for primitive variables with reduced precision, since the van Leer integrator does not directly use the prediction step in the final results. Such an approach saves up to ∼one-third of the device memory for hydrodynamics, while keeping the conservation law up to the full machine accuracy.

4. Verifications and Performance Tests

Several fundamental and standard tests of the Kratos code are presented in this section. Some of them are not only verifying the correctness and robustness of the algorithms involved but also used as the opportunity of testing the performance for the computing speed.

4.1. Convergence Tests

The fundamental tests of hydrodynamic methods and algorithms involve evolving the eigenfunctions of hydrodynamic equations, which include sound waves, contact waves, and shear waves. When periodic boundary conditions are applied, these eigenfunctions should ideally revert to their initial conditions after one kinematic crossing time, defined as $t_{\rm cross} \equiv l_{\rm box}/v$, where $l_{\rm box}$ represents the size of simulation domain (box), and v denotes the speed of the eigenmode along the specific direction of motion. To quantify the difference between the initial and evolved states, the normalized L_1 error is introduced as

$$L_1[q] \equiv \frac{\sum_i |q_i - q_i^0|}{N_{\rm cell}[\max\{q_i^0\} - \min\{q_i^0\}]}, \quad (6)$$

where $\{q_i^0\}$ and $\{q_i\}$ correspond to the values of the physical quantity q on the mesh before and after evolution, respectively. For clarity, the tests are initialized with background physical quantities $\rho_0 = 1$, $p_0 = 5/3$, and $\gamma = 5/3$. The 1D simulation

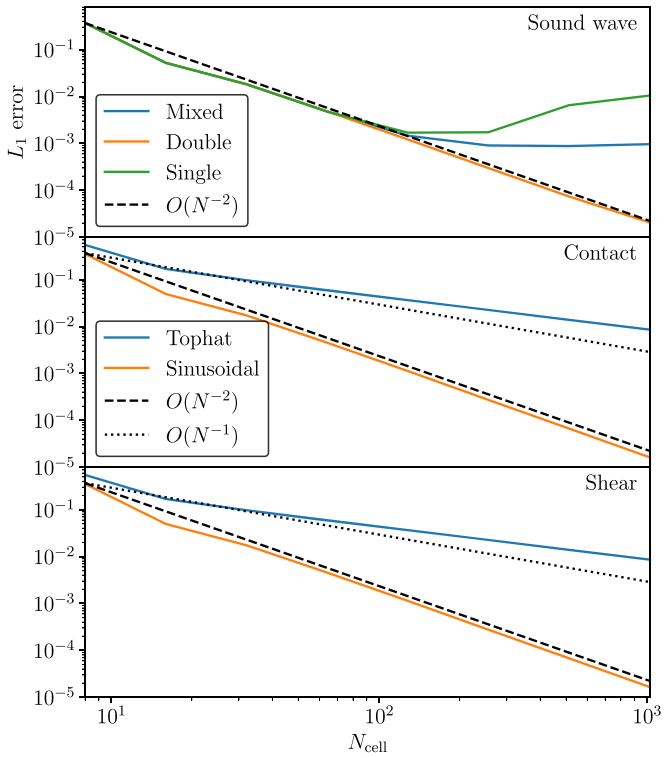


Figure 7. The L_1 errors in the convergence tests for the default hydrodynamic module in Kratos (see also Section 4.1). The top panel shows the sinusoidal sound wave tests, comparing the results with different computational precisions with the $L_1 \propto N^{-2}$ power law. The lower two panels exhibit the results for contact waves and shear waves with top-hat and sinusoidal initial conditions, respectively, using the mixed-precision methods (see also Section 3.2.2; note that full-double-precision results are almost identical to the mixed-precision ones, and are thus not illustrated).

domain, which has periodic boundaries, is set to have a size of $l_{\rm box} = 1$. Different types of waves are superimposed as perturbations $\{\delta q_i\}$ on this domain, using either a sinusoidal function $[\delta q_i = A \sin(2\pi x_i)]$ or a top-hat function ($\delta q_i = A$ if $x_i < 0.5$ and $\delta q_i = 0$ elsewhere). All tests are run from $t = 0$ to $t = 1$, covering a full period of motion. Tests with $v = -1$ are also carried out, generating identical results to confirm the reflection symmetry of the algorithms adopted.

The test results are illustrated in Figure 7. For the sound wave tests, it should be noted that the linearized sinusoidal sound wave serves as an approximate eigenfunction of the hydrodynamic equations. Such approximation necessitates sufficiently small amplitudes (numerical experiments indicate that $A \lesssim 10^{-6}$ is required) to ensure the wave evolving without physical distortions. However, when the amplitude is small and the resolution is high, the small contrasts between two adjacent cells significantly reduce the accuracy of reconstruction, which undermines the proper shapes of waves and the subsequent measurements of the L_1 errors. Consequently, for sound wave tests employing mixed- and full-single-precision methods, the amplitude is set at $A = 10^{-4}$, which prevents the L_1 errors from decreasing further with resolutions higher than $N_{\rm cell} \simeq 10^2$. In contrast, the full-double-precision case uses $A = 10^{-6}$ and achieves convergence up to $N_{\rm cell} > 10^3$, following a scaling rule of $L_1 \propto N_{\rm cell}^{-2}$. For contact waves (where only mass density varies) and shear waves (where only tangential velocity varies), a relatively larger amplitude ($A = 10^{-1}$) can be adopted, and the L_1 errors of mixed-precision methods are almost identical to the full-double-precision methods (omitted in this paper). Although the top-hat initial condition exhibits slower convergence than $L_1 \propto N_{\rm cell}^{-1}$, the sinusoidal waves still converge as $L_1 \propto N_{\rm cell}^{-2}$. This convergence behavior aligns with that of other existing numerical simulation codes utilizing the same hydrodynamic methods and algorithms (e.g., J. M. Stone et al. 2008, 2020, 2024).

4.2. Sod Shock Tubes

The shock tube problem serves as a canonical benchmark for assessing the accuracy and robustness of hydrodynamic solvers, as well as the integrity of the overall simulation infrastructure. While it is acknowledged that the presence of expansion fans within the shock tube scenario precludes the existence of exact analytic solutions, the derivation of semianalytic solutions remains a feasible and straightforward endeavor. In this context, a series of simulation results are presented and depicted in Figure 8, which correspond to a variety of initial configurations. These configurations are drawn from the standard test problems outlined in E. F. Toro (2009). It is important to highlight that the mixed-precision HLLC approximate Riemann solver has been deliberately selected for these tests, yielding results that demonstrate a consistent alignment with the analytic solutions across all examined cases.

Furthermore, extended tests based on the standard Sod shock tubes are also conducted, which traditionally consider fluid motion along the x-axis, by conducting additional tests that interchange the axes of fluid motion. The findings from these modified tests are found to be identical to those obtained from the standard x-axis tests. This congruence underscores the versatility of the finite volume scheme employed, which facilitates the calculation of fluxes through the preupdate primitive variables, thereby highlighting one of the key advantages of this approach.

4.3. Double Mach Reflection Test

The double Mach reflection tests, as canonical benchmarks for hydrodynamics simulations, are employed to rigorously evaluate the precision and dependability of simulation frameworks. This test, initially formulated by P. Woodward & P. Colella (1984), presents a scenario where a Mach 10 shock wave impinges upon an inclined plane, leading to complex interactions with the reflected shock wave and the emergence of a triple point. These dynamics yield a spectrum of hydrodynamic phenomena, including shock discontinuities and the generation of a high-velocity jet. The intricate flow structures that result are instrumental in assessing a code's capacity to accurately and robustly manage multidimensional discontinuities, particularly shock waves. For this test case, the initial conditions and boundary specifications are derived directly from P. Woodward & P. Colella (1984), with a slightly expanded spatial domain of $(x, y) \in [0, 4] \times [0, 1]$ and a resolution of 4096×1024. To accurately simulate the propagation of the inclined incident shock, a time-dependent boundary condition is implemented along the upper boundary, leveraging the p-boundary condition framework detailed in Section 3.1.1.

Figure 9 illustrates contour plots superimposed on a color-map representation of the solution at $t = 0.2$, juxtaposing mixed- and full-double-precision methods under identical conditions. It is evident that the two algorithms, despite their differing precision levels, exhibit near-perfect convergence,

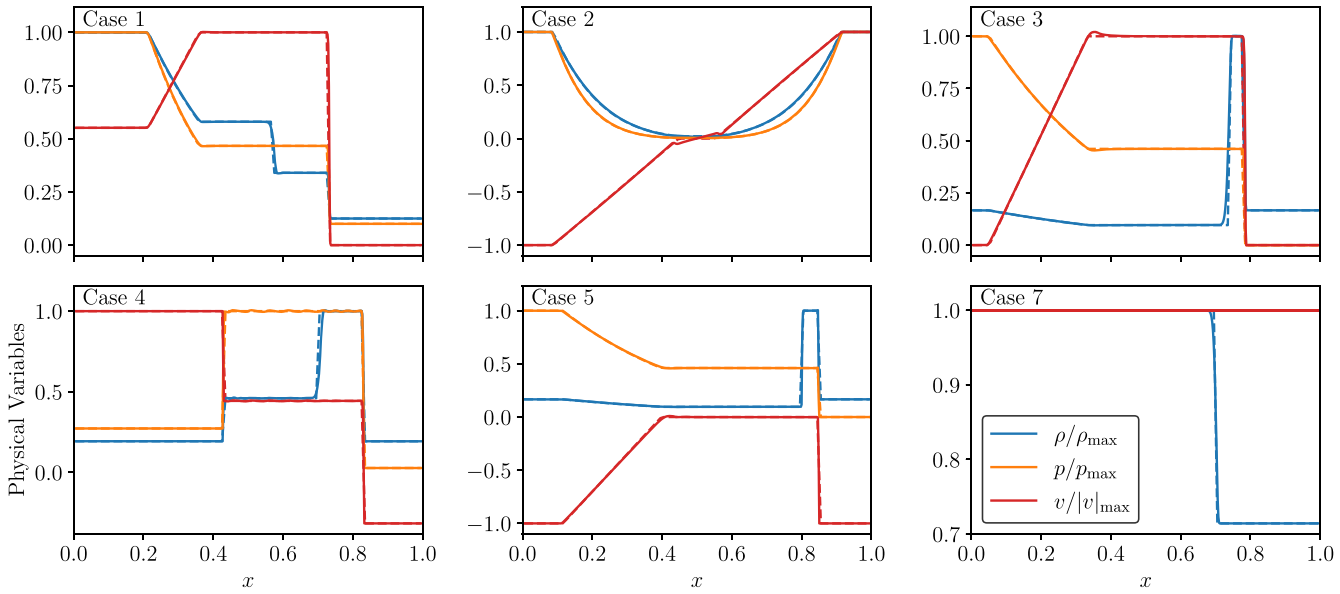


Figure 8. The 1D tests of Sod shock tubes, showing the results of cases 1–5 and 7 in E. F. Toro (2009) using the original setups and parameters. The exact semianalytic solutions are indicated by dashed lines, in the same color for the corresponding physical quantities of Kratos numerical calculation results. All calculations in these tests use mixed-precision methods.

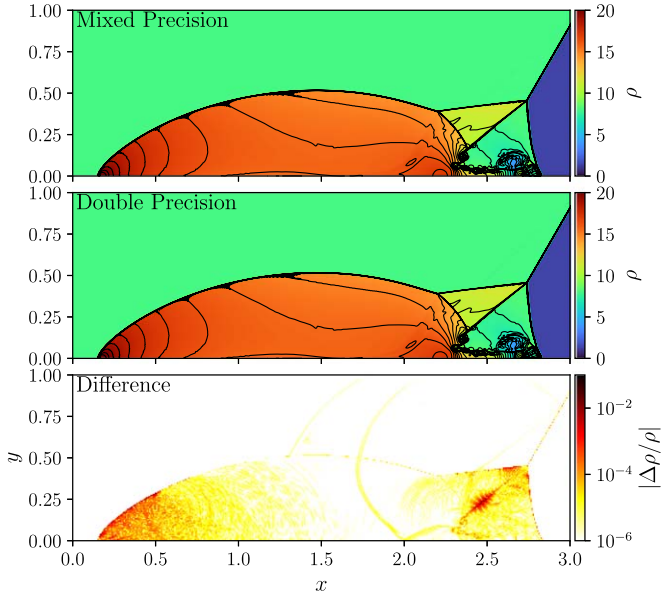


Figure 9. Results of double Mach reflection tests, using mixed-precision (top panel) and full-double-precision (middle panel) methods. The 30 contour lines starting from $\rho = 1.73$ to $\rho = 2.1$ are spaced evenly in ρ, indicating almost identical patterns on these panels. The bottom panel illustrate the relative differences of mass density comparing the top two panels.

with the maximum relative discrepancy not exceeding $\sim 10^{-2}$ as indicated by the mass density. These outcomes closely mirror Figure 4 in P. Woodward & P. Colella (1984), with the distinction that our results showcase sharper shock fronts attributable to higher-order spatial accuracy and a significantly enhanced resolution. In comparison with the analyses presented in M. Gittings et al. (2008) and J. M. Stone et al. (2020), the carbuncle-like instability near the jet's head is not entirely mitigated by the HLLC solver, which is known to introduce less numerical diffusion than the HLLE solver. Nevertheless, the near-identical behavior between mixed- and double-precision results at the jet head further substantiates the validity and robustness of the mixed-precision solver.

The performance assessments based on the double Mach reflection problem are summarized in Table 1, with 10^8-10^9 cells per second computational efficiency *per device* on contemporary GPUs. On GPUs for the consumer market (such as the NVIDIA RTX series and AMD 7900XTX), mixed-precision calculations run at speeds comparable to the full-single precision (around $\sim$70%−80%), and are almost $\gtrsim$5−7 times faster than the full-double-precision runs. These speeds are not proportional to the theoretical single- and double-precision floating-point computing efficiency of GPUs, as deeper profiling and timing tests of Kratos reveal that, even with relatively thorough optimization on the data exchange and cache or shared memory utilization (Section 3.2.1), a significant fraction of computing time is still occupied by the data exchange between the global graphics memory and the cache (not shown in this paper). On computing GPUs (such as Tesla A100 and MI100) with enhanced double-precision performance, the mixed-precision methods do not exhibit considerable advantages compared to full-double precision. The compatibility of Kratos on x64 and ARM CPUs is also confirmed and illustrated in Table 1. It is noted that the performance with HIP-CPU per physical CPU core is far below the other CPU codes on the same devices, by approximately $\sim$3−7 times (e.g., J. M. Stone et al. 2020). Because the HIP-CPU actually emulates GPUs using nonoptimized patterns (including the implementation of multi-threading dispatching algorithms and launching overheads), Kratos on HIP-CPU is implemented primarily for compatibility and code developing (especially debugging) considerations, rather than for performance purposes.

4.4. Liska–Wendroff Implosions

The Liska–Wendroff implosion test, as detailed in R. Liska & B. Wendroff (2003), serves as a benchmark for assessing the directional symmetry preservation of hydrodynamic simulation methods. This test is initiated with an initial condition that is similar to the Sod shock-tube scenarios, but with a diagonal orientation. A shock wave in the $\gamma = 1.4$ gas is generated by applying an under-

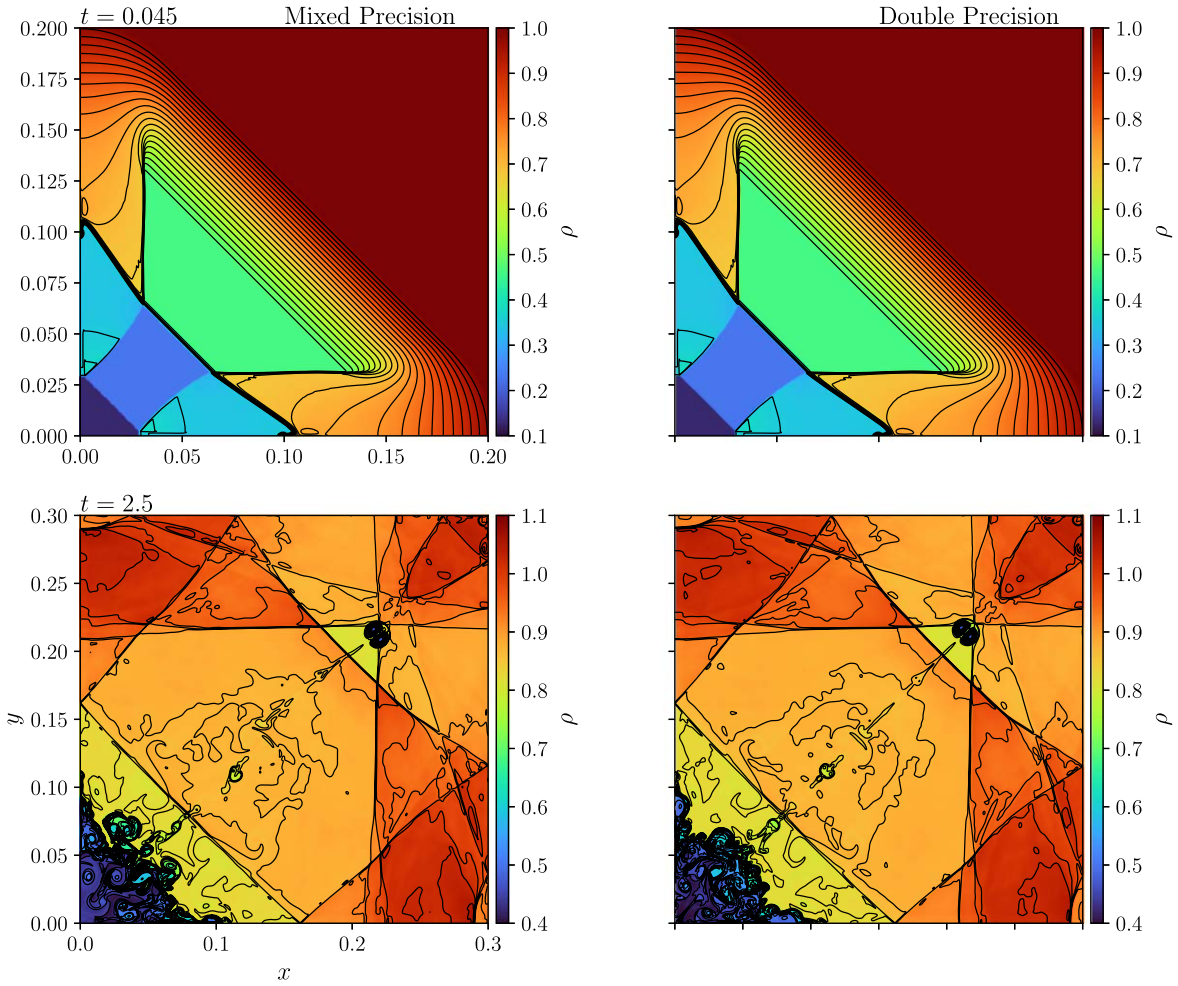


Figure 10. The Liska–Wendroff implosion tests results, showing $t = 0.045$ (upper row) and $t = 2.5$ (lower row), comparing the mixed-precision (left column) and full-double-precision (right column) methods. Contours span from $\rho = 0.35$ to $\rho = 1.1$, spaced evenly by $\delta\rho = 0.025$.

Table 1
Performance Test Results of Double Mach Reflections

Programming Models and Devices	Computing Speed with Precision (10^8 cell s^{-1})		
	Single	Mixed	Double
HIP-CPU[a]	…	…	…
AMD Ryzen 5950X[b]	…	…	0.15
Qualcomm Snapdragon 888[c]	…	…	0.0037
NVIDIA CUDA	…	…	…
RTX 2080TI	9.2	6.9	0.93
RTX 3090	14	10	1.4
RTX 4090	21	16	2.9
Tesla A100	16	14	8.3
AMD HIP	…	…	…
7900XTX	8.9	7.7	2.8
MI100	8.2	6.8	3.0

Notes. Presenting the average over 10^2 steps. All test cases are in 2D. For detailed setups see Section 4.3.

[a] Only double-precision results are concerned, as modern CPUs have almost the same single- and double-precision computing speeds.

[b] Utilizing all 16 physical cores.

[c] Using termux (https://termux.dev) on Android operating system, utilizing one major physical core. Compile-time optimization is turned *off* because of the software restrictions of TBB on ARM CPUs.

pressure region in the lower left corner of the domain,

$$(\rho, p, \boldsymbol{v}) = \begin{cases} (0.125, 0.14, 0), & x + y < 0.15; \\ (1, 1, 0), & \text{elsewhere}, \end{cases} \tag{7}$$

which is then reflected by the bottom and left boundaries. The initial discontinuity is set at a $\pi/4$ angle to the coordinate axes, with reflecting boundary conditions applied on all sides. The reflected shocks interact with other discontinuities, including contact discontinuities and additional shocks, leading to the development of fingerlike structures via the Richtmeyer–Meshkov instability along the direction of the initial normal vector. The direction of the jet serves as a critical indicator of whether the simulation method maintains reflective symmetry to machine precision. In some early hydrodynamic simulation systems that employed directionally split algorithms, the jet's impact at the lower left corner might not align precisely with the corner, resulting in vortices that deviate significantly from the diagonal line. This deviation highlights the importance of directional symmetry in accurately capturing the dynamics of the system.

Figure 10 illustrates the density profiles at $t = 0.045$ and $t = 2.5$, comparing the results obtained from mixed-precision and double-precision methods. The double-precision results exhibit good symmetry across the diagonal at $t = 2.5$, with the exception of the lower left corner, which is prone to strong instabilities and

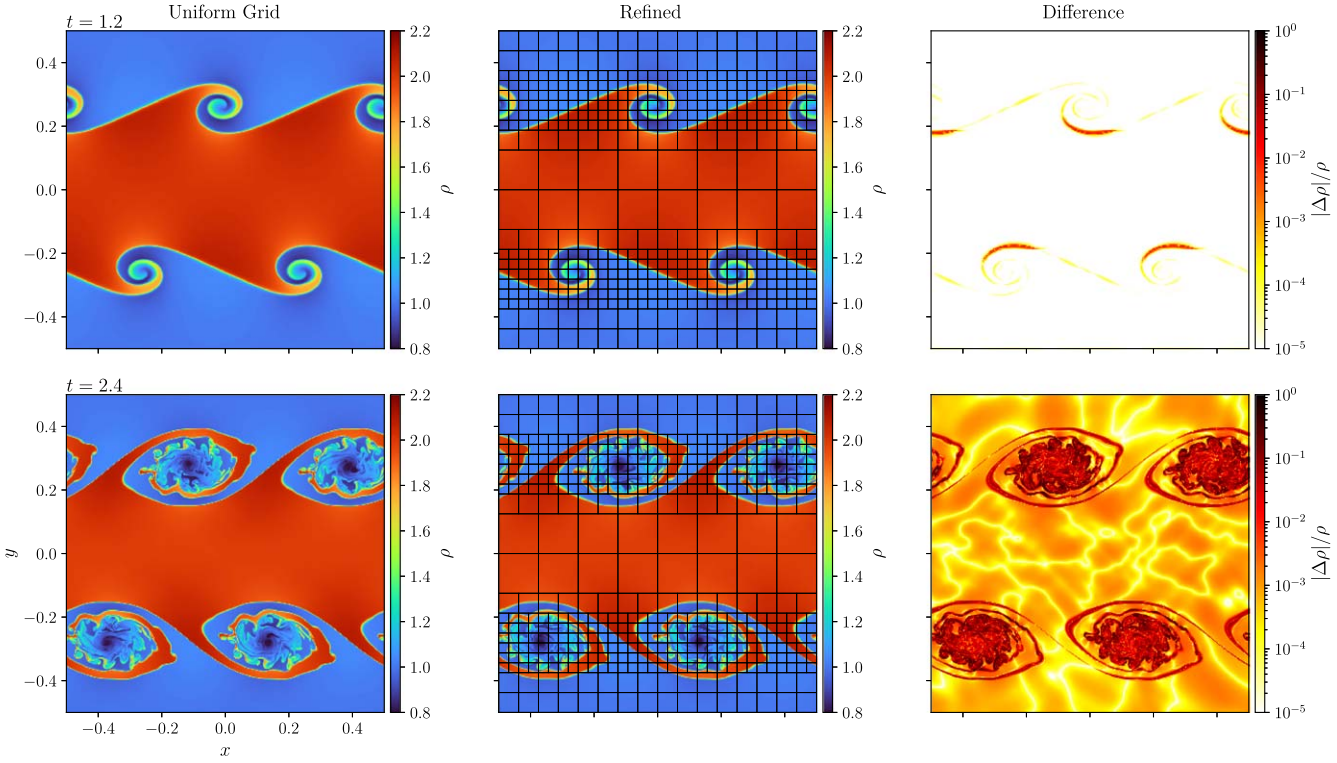


Figure 11. The Kelvin–Helmholtz instability tests results, comparing the $t = 1.2$ (upper row) and $t = 2.4$ (lower row) for the uniform grid (left column) and static mesh-refinement (SMR; middle column) results for the relative difference (right panel) in mass density. The block boundaries are indicated in the panels for the SMR simulation, showing the mesh-refinement results (note that each block has the same number of cells).

turbulence. The mixed-precision results also maintain a large degree of reflection symmetry; however, the wake of the jet-induced vortices shows a noticeable departure from perfect symmetry when visualized with contour plots. It is important to note that the last significant digits in floating-point calculations on GPUs are not guaranteed to be identical for both single and double precision, and the associated errors are inherently undetermined (e.g., NVIDIA 2025). Amplified by chaotic motions, such random errors eventually lead to the symmetry breaking. Before employing mixed-precision methods in practical applications, it is crucial to ascertain whether strict reflection symmetry is necessary. Additionally, it should be recognized that this type of discrete symmetry breaking, caused by slightly differing results from separate calculations, does not affect continuous symmetries, such as the conservation laws of mass and momentum.

4.5. Kelvin–Helmholtz Instabilities

The tests on Kelvin–Helmholtz instabilities are some of the "standard procedures" for many numerical simulation systems. Such tests, similar to those in D. Lecoanet et al. (2016), are conducted on $\gamma = 1.4$ gases with initial conditions,

$$\begin{aligned} \rho &= \rho_0 + \Delta\rho \tanh\left(\frac{|y - y_0|}{L}\right), \\ v_x &= v_{x0} \tanh\left(\frac{|y - y_0|}{L}\right), \\ v_y &= v_{y0} \cos(kx) \exp\left[-\frac{(y - y_0)^2}{2\sigma^2}\right], \end{aligned} \tag{8}$$

where ρ_0, v_{x0}, and v_{y0} are the reference values for density and x, y velocity components, $\Delta\rho$ is the difference in density across the shearing layer with thickness L, k is the velocity perturbation wavenumber, and σ is the thickness of the perturbation region. This paper chooses the parameters $L = 0.01$, $y_0 = 0.25$, $\rho = 1.5$, $\Delta\rho = 0.5$, $v_{x0} = 0.5$, $v_y = 0.01$, $k = 4\pi$, and $\sigma = 0.2$. These 2D simulations are carried out within a spatial domain of $x, y \in [-0.5, 0.5] \times [-0.5, 0.5]$. The uniform grid simulations employ a resolution of $\Delta x = 1/2048$ across the entire domain, whereas the mesh-refinement approach ensures this resolution is maintained in the regions $|x| \in (3/16, 3/8)$ through the application of two levels of static mesh refinement. Periodic boundary conditions are applied to the x boundaries, and reflective conditions are employed elsewhere.

The results of these tests are illustrated in Figure 11, which compares the outcomes from uniform grid simulations with those from simulations utilizing mesh refinement. At $t = 1.2$, the upper row of the figure illustrates results that are nearly indistinguishable between the two methods, with a relative error not exceeding 5×10^{-2}. The minor differences observed are primarily localized near regions of density discontinuities, likely due to subtle structural displacements. At $t = 2.4$, the patterns remain largely consistent, yet the quantitative differences between the uniform grid and mesh-refinement results are more pronounced, reaching up to $\sim 10^{-1}$. These discrepancies are also attributed to structural displacements, as evidenced by a direct comparison of the uniform grid and mesh-refinement panels. It is important to note that simulations employing mesh refinement may still exhibit variations from those with high-resolution uniform grids, particularly in the damping of short-wavelength modes within unrefined, low-resolution areas. The kinetic diffusivity parameter, which is influenced by numerical methods, scales with $\eta \propto \Delta x^2/\Delta t$. This phenomenon has also been observed in various studies, and is further supported by the work of J. M. Stone et al. (2020).

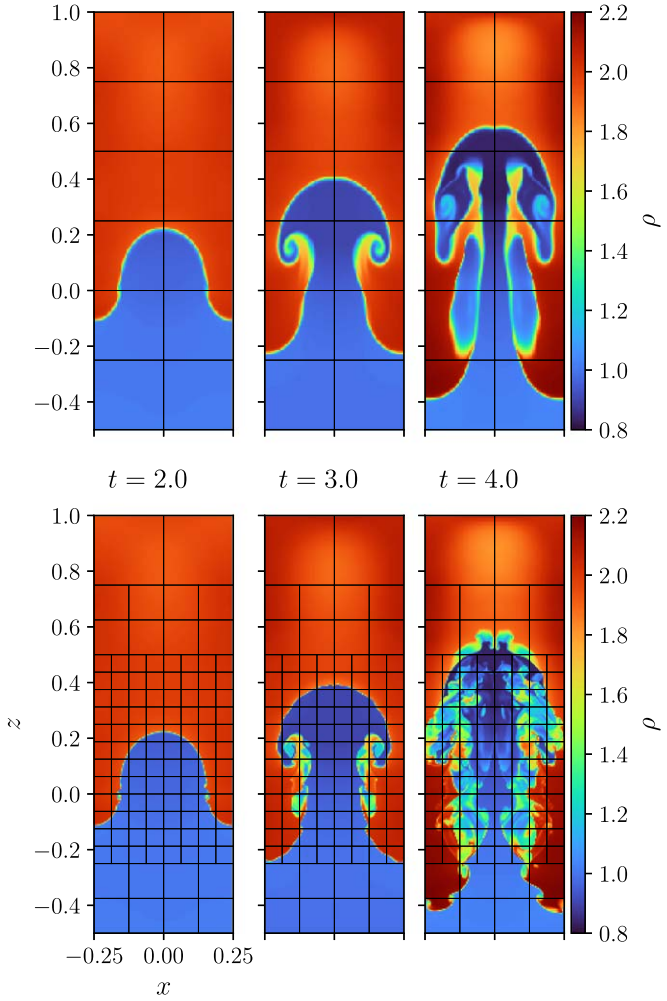


Figure 12. The Rayleigh–Taylor instability tests, showing $t = 2$, 3, and 4 results in three columns for the simulations on uniform grid (top row) and with mesh refinement (bottom row). Similar to Figure 11, the block boundaries are indicated to illustrate the mesh-refinement layouts.

4.6. Rayleigh–Taylor Instabilities

The Rayleigh–Taylor instability (RTI) test, a standard benchmark for evaluating the efficacy of numerical methods in multidimensional hydrodynamics, is further detailed in, e.g., R. Liska & B. Wendroff (2003) and D. Lecoanet et al. (2016). This test is useful for assessing how numerical schemes behave in the presence of complex fluid interactions. This test also examines the performance of the code under static mesh refinement, as the regions where significant hydrodynamic features are anticipated to emerge can be identified with relative ease a priori.

The simulated domain covers $(x, y, z) \in [-1/4, 1/4] \otimes [-1/4, 1/4] \otimes [-1/2, 1]$ with $\gamma = 1.4$ gas. While the uniform grid test employs a grid spacing of $\Delta x = 1/256$, the mesh-refinement scenario focuses on a refined region of $(x, y, z) \in [-1/4, 1/4] \otimes [-1/4, 1/4] \otimes [-1/2, 1]$ to the second level, achieving a grid spacing of $\Delta x = 1/1024$. Reflecting boundary conditions are implemented on all physical boundaries. Initially, the density is set to $\rho = 2$ for $z > 0$ and $\rho = 1$ for $z \leqslant 0$. The pressure is determined based on the vertical gravitational acceleration $g_z = -0.5$ (negative indicating the downward direction of gravity), ensuring a hydrostatic equilibrium calibrated at $p_{z=0} = 2.5$. An initial velocity perturbation is introduced as $v_z = 10^{-2}\cos(\boldsymbol{k} \cdot \boldsymbol{x})$, where $\boldsymbol{k} \equiv (4\pi, 4\pi, 3\pi)$ represents the perturbation wavenumber.

Table 2
Performance Test Results Based on the Rayleigh–Taylor Instability Simulations

Programming Models and Devices	Computing Speed with Precision[a] (10^8 cell s^{-1})		
	Single	Mixed	Double
HIP-CPU	…	…	…
AMD Ryzen 5950X[b]	…	…	(0.15)
NVIDIA CUDA	…	…	…
RTX 3090	(8.6, 14.7)	(6.2, 11.2)	(0.61, 1.1)
RTX 4090	(12.4, 17.7)	(10.1, 16.3)	(1.3, 2.4)
Tesla A100	(8.3, 14.6)	(8.3, 12.9)	(4.7, 8.2)
AMD HIP	…	…	…
7900XTX	(3.6, 5.7)	(3.4, 5.5)	(0.94, 1.86)
MI100[c]	(3.7)	(3.6)	(2.1)

Notes. Presenting the average over 10^2 steps. Results are presented as groups of numbers (s_1, s_2), where s_1 is for the single-device speed, and s_2 for the dual-device one. All test cases are in 3D (see Section 4.6).
[a] Some devices involved in Table 1 are not applicable in these tests.
[b] Dual-device tests not applicable.
[c] Tests are not conducted on dual-device MI100 systems due to technical restrictions.

The outcomes of the tests, encompassing mesh-refinement conditions, are depicted in Figure 12, presenting a comparative analysis between simulations with and without mesh refinement at various evolutionary times. It is important to note that, due to the absence of explicit viscosity or surface tension, the emergent shear instabilities parasitic on the RTI patterns are theoretically expected to grow at nearly all wavelengths. The actual results are, therefore, affected by the numerical diffusivities, which are proportional to $\eta \propto \Delta x^2/\Delta t$. This leads to significant differences when comparing mesh-refinement results with those obtained using low-resolution uniform grids. With an equivalent resolution of $\Delta x = 1/1024$ on the refined mesh, the nonlinear fine structures are more pronounced due to reduced numerical diffusivities, while the overall "mushroom" structures remain quantitatively similar for both scenarios.

The performance metrics of Kratos in these tests are summarized in Table 2. As the RTI tests are conducted in 3D, the time consumption due to the flux calculations on the third dimension nominal reduces the speed (in cells per second) by around one-third when compared to the 2D results in Table 1. One can also observe by comparing the single-device speed with dual-device speed (the first and second numbers of each data entry in the table) that the cross-device parallelization performance is good with two devices other than RTX 4090 (up to ~90% the theoretical performance). This indicates that the workflow design (see also Section 3 and Figure 3) is successfully optimizing out the communication overheads. When using the RTX 4090 GPUs, however, the scalability appears to be worse, as the high single-GPU performance partially fails the workflow in hiding the communication operations behind computations. It is noted that modern CPU-based codes carry out hydrodynamic simulations with the same temporal and spatial accuracy (also using the slope-limited PLM reconstruction, the HLLC Riemann solver, and the second-order Heun integrator) and same mesh-refinement configurations at a speed of $\sim 3 \times 10^6$ cells^{-1} per CPU core (e.g., Athena++; see J. M. Stone et al. 2020), or

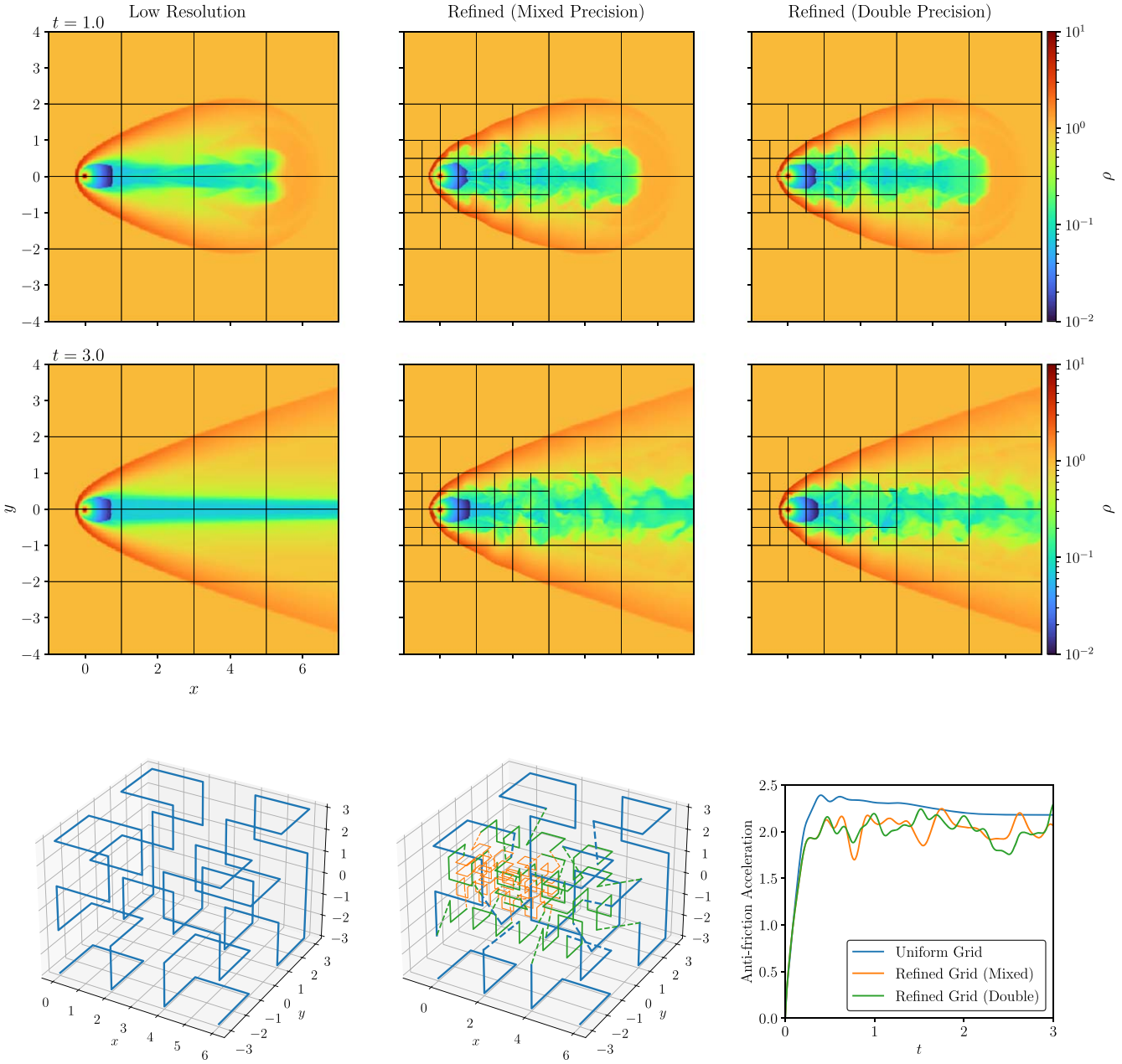


Figure 13. Results of the tests on colliding outflows and ambient gases, showing the mass density profiles at $t = 1$ (top row) and $t = 2$ (middle row) for low resolution ($\Delta x = 1/32$; left column), static mesh-refinement (SMR) with mixed-precision (equivalent $\Delta x = 128$; middle column), and SMR with full-double-precision (right column) cases. Block boundaries are indicated in related panels to illustrate the mesh layouts. The bottom row illustrates the Hilbert curves generated for the job dispatching and load balancing for the uniform grid (left panel) and SMR (middle panel) simulations, where the lines connecting blocks on different levels are shown in different colors, and cross-level connections are indicated by dashed lines. The evolution history for the acceleration on the central object of the three different simulations (lower right panel).

$\sim 3 \times 10^8$ cells^{-1} per 128-core computing node given perfect scalability. This speed is equivalent to ~30% of one RTX 4090 GPU, and one may have noticed that each node can be equipped with multiple (up to eight) GPUs, leading to ~20 times computing performance improvement per computing node.

4.7. Colliding as Outflow and Ambients

A traditional yet highly relevant scenario, which could showcase the performance and capabilities of the simulation, involves interactions between stellar outflows and ambient gas flows induced by the relative motion of stars and their surrounding medium. This scenario not only is of historical significance but also gains contemporary relevance in the dynamical studies of stars and compact objects, as highlighted in recent literatures (e.g., X. Li et al. 2020; L. Wang & X. Li 2022).

These tests are conducted for $\gamma = 1.4$ gas, within a cubic spatial domain defined by the coordinates $(x, y, z) \in [-1, 7] \otimes [-4, 4] \otimes [-4, 4]$. At the left boundary along the x-axis, an inflow is introduced with a velocity of $v_{\rm in} = 5$, a density of $\rho_{\rm in} = 1$, and a pressure of $p_{\rm in} = 1$. At the coordinate origin $(x, y, z) = (0, 0, 0)$, a spherical source with a radius $r = 0.1$ is established to emit gas at a rate of $\dot{m} = 13$ units of mass per unit time with a radial velocity of $v_r = 10$. The outcomes of these tests are depicted in Figure 13, where three distinct cases are compared:

1. a uniform grid with $\Delta x = 1/32$ with full-double precise methods;

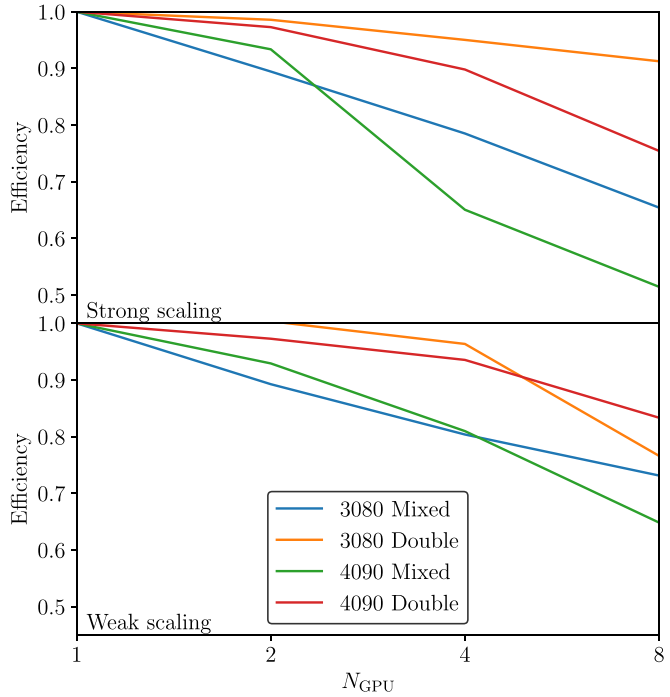


Figure 14. Scaling performance on multiple GPUs on different types of devices (RTX 3080 and RTX 4090), with mixed-precision and full-double-precision methods respectively. Tests are based on the outflow–incoming flow interactions (see Section 4.7 and Figure 13). The upper panel shows the strong scaling (with the same total computation load), and the lower panel is for the weak scaling (with the same computation load per device).

2. a refined grid with an equivalent resolution $\Delta x = 1/128$ within the $(x, y, z) \in [-1, 3] \otimes [-1, 1] \otimes [-1, 1]$ region, using mixed-precision methods (Section 3.2.2);
3. same as 2, but using full-double-precision methods.

The comparisons are made at two temporal snapshots, $t = 1$ and $t = 3$. At $t = 1$, the semiquantitative similarities among the three cases are apparent. The first case lacks instabilities due to higher damping from lower resolution, while the latter two cases, by incorporating mesh refinement, display clear signs of growing instabilities. By $t = 3$, the divergence between the latter two cases becomes more apparent, as evidenced in the lower right panel illustrating the net acceleration. The reflection symmetry across the plane $y = 0$ is broken in both refined cases, even with the utilization of full-double precision, despite commencing from identical initial conditions that strictly maintain reflection symmetry. This asymmetry, as discussed in Section 4.4, originates from the intricacies of floating-point calculations on heterogeneous devices. With hydrodynamic instability modes unattenuated at higher resolutions, the disruption of symmetry and the ensuing chaotic turbulent evolution are anticipated phenomena.

The lower left and lower center panels in Figure 13 exhibit the application of Hilbert curves in scenarios with uniform and refined meshes, respectively. It is evident that, with the L-system employed in the mesh-refinement framework (Section 2.3.2), the Hilbert curve is properly generated within the refined region through recursion, and task distribution is executed accordingly. A few scalability tests are also carried out under this scenario, as summarized by Figure 14. Similar to the data presented in Section 4.6 and Table 2, the strong and weak scalings of Kratos on multiple devices are good when the single-device computation speed is relatively slow ($\lesssim 3 \times 10^8$ cells per second per device), and the results start to deteriorate with faster computing modes (e.g., mixed precision with RTX 4090) the issue of communication overhead (by "not perfectly hiding communication behind computation"). It is noted that the limitations on multi-GPU scalability are not primarily due to the host–device interface. Instead, the fundamental issue lies in the driver-level prohibition of the GPU–GPU interface for consumer-level GPUs, which forces code developers to manage communication in special ways, including those detailed in this work. Although Kratos does support GPU-to-GPU direct communication when it is available, this work deliberately avoids focusing on such optimal cases to prevent misleading readers. In addition, cross-node communication indeed results in the deterioration of parallelization on larger scales, with the extent of this deterioration largely dependent on the hardware setup. However, such deterioration saturates whenever the number of nodes employed is greater than 1. Moreover, introducing more microphysical modules, which is the core objective of constructing the entire Kratos system, considerably alleviates the issue of being unable to "hide" communication costs. These modules are part of subsequent manuscripts currently being prepared and fall outside the scope of this present one.

5. Summary and Forthcoming Works

This paper describes the Kratos Framework, a novel GPU-based simulation system tailored for astrophysical simulations on heterogeneous devices. Kratos is designed to harness modern GPUs, providing a flexible and efficient platform for simulating a broad spectrum of astrophysical phenomena. Its fundamental infrastructures rely on abstraction layers for devices, a multiprocessing communication model, and a mesh-management system. These building blocks are designed for heterogeneous computing environments (especially GPUs) consistently, preparing for the supports for various physical modules, including hydrodynamics (which is elaborated in this paper), and thermochemistry, magnetohydrodynamics, self-gravity, and radiative processes in forthcoming papers.

Kratos is constructed for the CUDA, HIP, and similar GPU programming models, employing abstraction layers to insulate algorithms from programming model disparities, thereby optimizing performance and ensuring compatibility across different GPUs without modifying the actual implementations of algorithms. The basic mesh structures of Kratos utilize 2^d-trees for structured mesh management, with mesh refinement implemented recursively. Job dispatching and load balancing, which can involve user-defined schemes, use Hilbert curves based on L-systems by default. Mesh structures are designed to be separated from physical modules, so that involving multiple physical modules can be naturally accomplished. The multiprocessing communication model is implemented for device-to-device communications that allow the GPU stream to be involved, thus optimized for the computation on systems equipped with consumer-level GPUs.

The hydrodynamics module of Kratos is constructed within the Godunov framework, and the default implementations of submodules are slope-limited PLM reconstruction, HLLC Riemann solver, and Heun time integrator. Optimizations for heterogeneous devices include variable conversion, boundary conditions, flux calculation, and time integration. The module employs mixed-precision methods to maintain conservation laws with machine-level accuracy while approximating fluxes.

Verification and performance tests include Sod shock tubes, double Mach reflection test, Liska–Wendroff implosion, Kelvin–Helmholtz instabilities, Rayleigh–Taylor instabilities, and colliding stellar outflows and ambients. These tests validate algorithms, robustness, and performance of Kratos with results aligning with analytic solutions and demonstrating the effectiveness of the hydrodynamics module under various conditions.

There are, admittedly, still prospective caveats and issues to be solved in the current Kratos implementation, which are also postponed to future works. For example, the prohibition of direct data exchanges between consumer-level GPUs significantly worsen the scalability of Kratos, especially when the computing speed on a single device is too fast to cover the communication costs. However, this issue is naturally solved by using computing-specific GPUs that allow faster direct communications. The inclusion of computational heavy multiphysics modules (such as real-time thermochemistry) will also dwarf the communication overhead. With various modules for multiple physical mechanisms being implemented, such as magnetohydrodynamics solver, realtime nonequilibrium thermochemistry solver, multigrid Poisson equation solver, and particle-based solver (to be elaborated by forthcoming papers), Kratos is expected to evolve into a comprehensive system aiming at astrophysical simulations and beyond.

Acknowledgments

L.W. acknowledges the support in computing resources provided by the Kavli Institute for Astronomy and Astrophysics in Peking University. The author thanks the following colleagues for helpful discussions and useful suggestions (alphabetical order): Xue-Ning Bai, Renyue Cen, Zhuo Chen, Can Cui, Ruobin Dong, Jeremy Goodman, Luis C. Ho, Xiao Hu, Kohei Inayoshi, Ming Lv, Mordecai Mac-Low, Kengo Tomida, Zhenyu Wang, Haifeng Yang, and Yao Zhou.

Code availability. As Kratos is still being actively developed, the author will only provide the code upon request and collaboration at this moment. While several important modules that are already mature have already been adopted and made public along with S. Wang et al. (2025), a more complete version of Kratos will be available publicly after further and deeper debugs are accomplished.

ORCID iDs

Lile Wang https://orcid.org/0000-0002-6540-7042

References

Canning, P., Cook, W., Hill, W., Olthoff, W., & Mitchell, J. C. 1989, in Proc. of the Fourth Int. Conf. on Functional Programming Languages and Computer Architecture, FPCA 1989, ed. J. E. Stoy (New York: Association for Computing Machinery), 273
Coplien, J. O. 1995, C++ Rep., 7, 24
Gittings, M., Weaver, R., Clover, M., et al. 2008, CS&D, 1, 015005
Grete, P., Glines, F. W., & O'Shea, B. W. 2021, IEEE Trans. Parallel Distrib. Syst., 32, 85
Kestener, P. 2017, Ramses-GPU: Second Order MUSCL-Handcock Finite Volume Fluid Solver, Astrophysics Source Code Library, ascl:1710.013
Lecoanet, D., McCourt, M., Quataert, E., et al. 2016, MNRAS, 455, 4274
Lesur, G. R. J., Baghdadi, S., Wafflard-Fernandez, G., et al. 2023, A&A, 677, A9
Li, X., Chang, P., Levin, Y., Matzner, C. D., & Armitage, P. J. 2020, MNRAS, 494, 2327
Liska, R., & Wendroff, B. 2003, SJSC, 25, 995
Lv, A., Wang, L., Cen, R., & Ho, L. C. 2024, ApJ, 977, 274
NVIDIA 2024, CUDA C++ Programming Guide v12.8, NVIDIA Corporation, https://docs.NVIDIA.com/cuda/cuda-c-programming-guide
NVIDIA 2025, Floating Point and IEEE 754 Compliance for NVIDIA GPUs v12.8, NVIDIA Corporation, https://docs.nvidia.com/cuda/floating-point/index.html
Schive, H.-Y., ZuHone, J. A., Goldbaum, N. J., et al. 2018, MNRAS, 481, 4815
Schneider, E. E., & Robertson, B. E. 2015, ApJS, 217, 24
Stone, J. M., Gardiner, T. A., Teuben, P., Hawley, J. F., & Simon, J. B. 2008, ApJS, 178, 137
Stone, J. M., Mullen, P. D., Fielding, D., et al. 2024, arXiv:2409.16053
Stone, J. M., Tomida, K., White, C. J., & Felker, K. G. 2020, ApJS, 249, 4
Toro, E. F. 2009, Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction (3rd ed.; Dordrecht: Springer)
Trott, C. R., Lebrun-Grandié, D., Arndt, D., et al. 2022, IEEE Trans. Parallel Distrib. Syst., 33, 805
Wang, L., & Li, X. 2022, ApJ, 932, 108
Wang, S., Wang, L., & Dong, S. 2025, ApJS, 276, 40
Woodward, P., & Colella, P. 1984, JCoPh, 54, 115
Zhang, W., Almgren, A., Beckner, V., et al. 2019, JOSS, 4, 1370